



Parul[®]
University

NAAC A++

FACULTY OF ENGINEERING & TECHNOLOGY
BACHELOR OF TECHNOLOGY

Design & Analysis of Algorithm

(303105219)

5th SEMESTER

ARTIFICIAL INTELLIGENCE &
MACHINE LEARNING

Laboratory Manual

CERTIFICATE

***This is to certify that Ms.Myadarapu Adithyasai with enrollment no.
2403031460510 has successfully completed his laboratory experiments in
Design & Analysis of Algorithm (303105219) from the department of
Artificial Intelligence & Machine Learning during the academic
year 2026-27.***



Date of Submission:.....

Staff In charge:.....

Head of Department:.....

INDEX

SR. NO	PROGRAM	PAGE. NO	DATE OF PERFORMANCE	DATE OF SUBMISSION	MARKS	SIGN
1	write a program to determine whether the given number is Prime or not.					
2	Given a sorted array and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.					
3	There are N children standing in a line with some rating value. You want to distribute a minimum number of candies to these children such that: Each child must have at least one candy. The children with higher ratings will have more candies than their neighbours. You need to write a program to calculate the minimum candies you must give.					
4	There is a new barn with N stalls and C cows. The stalls are located on a straight line at positions $x_1, x_2, \dots, x_N$ ($0 \leq x_i \leq 1,000,000,000$). We want to assign the cows to the stalls, such that the minimum distance between any two of them is as large as possible. What is the largest minimum distance?					
5	Given an undirected graph with V vertices and E edges, check whether it contains any cycle or not.					

6	There are n servers numbered from 0 to n-1 connected by undirected server-to-server connections forming a network where $connections[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network. A critical connection is a connection that, if removed, will make some servers unable to reach some other servers. Return all critical connections in the network in any order.					
7	Given a grid of size N x M (N is the number of rows and M is the number of columns in the grid) consisting of 'O's (Water) and '1's(Land). Find the number of islands.					
8	Given a grid of dimension N x M where each cell in the grid can have values 0, 1, or 2 which has the following meaning: 0: Empty cell 1: Cells have fresh oranges 2: Cells have rotten oranges We have to determine what is the minimum time required to rot all oranges. A rotten orange at index [i,j] can rot other fresh oranges at indexes [i-1,j], [i+1,j], [i,j-1], [i,j+1] (up, down, left and right) in unit time'					
9	Given two strings str1 and str2 and the allowed operations on strings are: Insert, Remove, Replace, All of the above operations are of equal cost.					

10	Minimum Path Sum" says that given a $n \times m$ grid consisting of non-negative integers and we need to find a path from top-left to bottom right, which minimizes the sum of all numbers along the path.					
11	Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num.					
12	There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., <code>grid[0][0]</code>). The robot tries to move to the bottom-right corner (i.e., <code>grid[m-1][n-1]</code>). The robot can only move either down or right at any point in time. Given the two integers m and n, return the number of possible unique paths that the robot can take to reach the bottom-right corner.					

Practical-1

Aim: To write a program that checks whether a given number is prime or not.

Algorithm:

1. Start the program.
2. Read an integer n from the user.
3. If n is less than or equal to 1, display "**Not a Prime Number**" and stop.
4. Initialize a variable flag = 0.
5. Repeat from i = 2 to $\sqrt{n}$:
 - o If $n \% i == 0$:
 - o Set flag = 1.
 - o Exit the loop.
6. If flag == 0, display "**Prime Number**".
7. Otherwise, display "**Not a Prime Number**".
8. Stop.

Expected Input/Output (Test Cases)

<i>Test Case</i>	<i>Input</i>	<i>Expected Output</i>
1	7	7 is a Prime Number
2	10 29	10 is Not a Prime Number
3		29 is a Prime Number

CodeImplementation:

```
import java.util.Scanner;
public class Main {
    public static boolean isPrime(int n) {
        if(n <= 1) return false;           // 1. Correctly handles numbers less than or equal to 1.
        if(n == 2) return true;            // 2. Correctly identifies 2 as a prime number.
        if(n % 2 == 0) return false;       // 3. Eliminates all other even numbers instantly.

        //4. Loops only through odd numbers up to the square root of n.

        for (int i = 3; i <= Math.sqrt(n); i+=2){
```

```
        if(n%i==0) return false;
    }
    return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int n = sc.nextInt();

    if (isPrime(n))
        System.out.println(n + " is a Prime number.");
    else
        System.out.println(n + " is NOT a Prime number.");

    sc.close(); // 5. Properly closes the Scanner resource.
}
}
```

Time Complexity

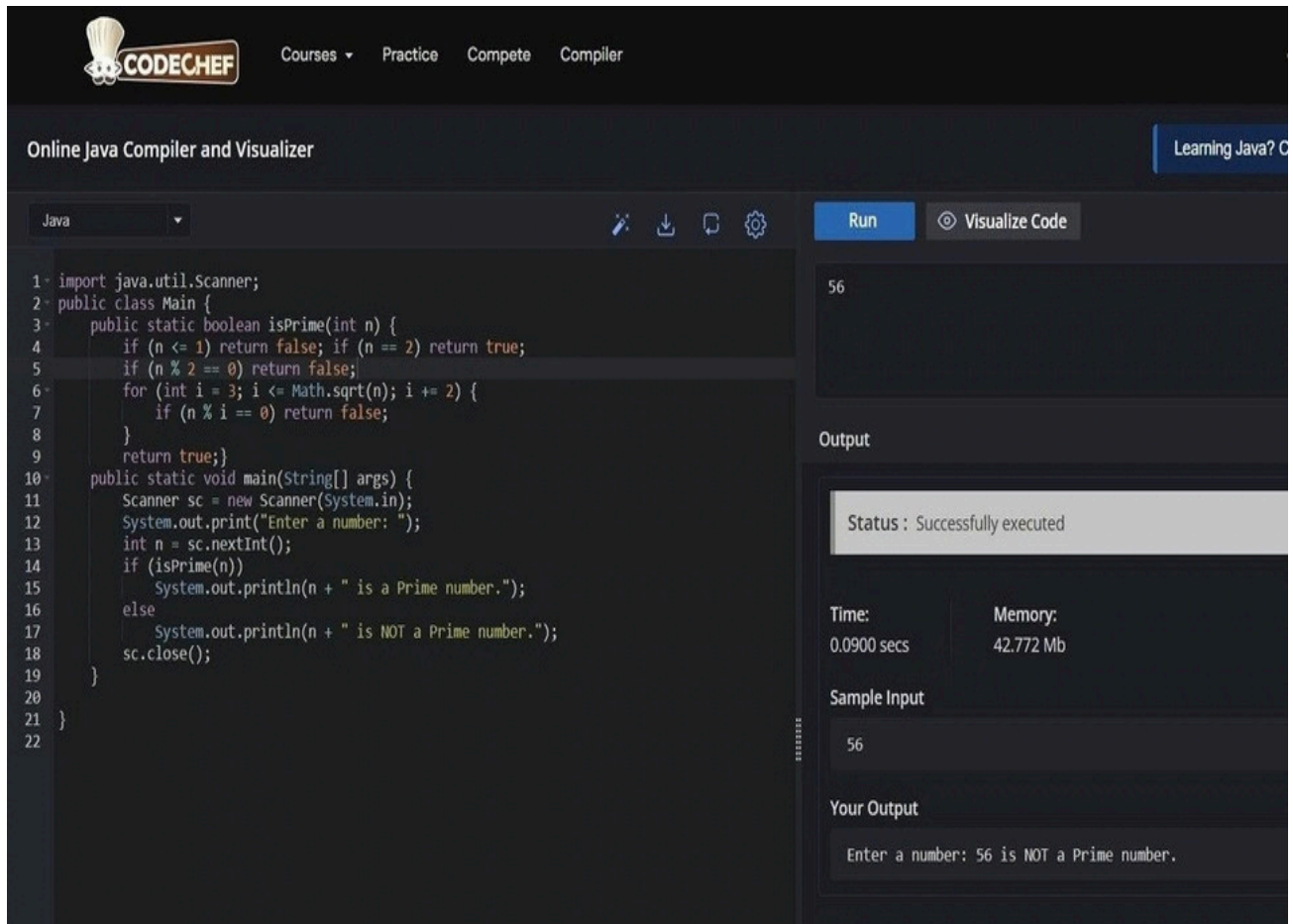
Best Case: $O(1)$ (when $n \leq 1$ or the first divisor is found, e.g., an even number greater than 2)

Worst Case: $O(\sqrt{n})$ (when the number is prime and all possible divisors up to $\sqrt{n}$ are checked)

Space Complexity

$O(1)$: (Constant space, as only a few variables are used regardless of the input size.)

Input/Output:



Online Java Compiler and Visualizer

Java

```
1 import java.util.Scanner;
2 public class Main {
3     public static boolean isPrime(int n) {
4         if (n <= 1) return false; if (n == 2) return true;
5         if (n % 2 == 0) return false;
6         for (int i = 3; i <= Math.sqrt(n); i += 2) {
7             if (n % i == 0) return false;
8         }
9         return true;
10    }
11    public static void main(String[] args) {
12        Scanner sc = new Scanner(System.in);
13        System.out.print("Enter a number: ");
14        int n = sc.nextInt();
15        if (isPrime(n))
16            System.out.println(n + " is a Prime number.");
17        else
18            System.out.println(n + " is NOT a Prime number.");
19        sc.close();
20    }
21 }
22
```

Run Visualize Code

56

Output

Status : Successfully executed

Time: 0.0900 secs Memory: 42.772 Mb

Sample Input

56

Your Output

Enter a number: 56 is NOT a Prime number.

Practical-2

Aim: To find the index of a target value in a sorted array, or the specific index where it would be sequentially inserted to maintain sorted order.

Algorithm:

1. Start the program.
2. Read the size of the sorted array n.
3. Read the sorted array elements.
4. Read the target value.
5. Initialize low = 0 and high = n - 1.
6. Repeat while low <= high:
 - o Find the middle index:
 $mid = low + (high - low) / 2$
 - o If array[mid] == target, return mid.
 - o Else if array[mid] < target, set low = mid + 1.
 - o Else, set high = mid - 1.
7. If the target is not found, return low as the insertion position.
8. Display the index.
9. Stop.

Expected Input/Output (Test Cases)

<i>Test Case</i>	<i>Input Array</i>	<i>Target</i>	<i>Expected Output</i>
1	[1, 3, 5, 6] [1,	5	Index = 2
2	3, 5, 6] [1, 3,	2	Index = 1 (Insertion Position)
3	5, 6]	7	Index = 4 (Insertion Position)

Code Implementation: for binary search

```
import java.util.Scanner;

public class SearchInsertPosition {

    public static int searchInsert(int[] arr, int target) {

        int low = 0, high = arr.length - 1;
        while (low <= high) {
```

```
int mid=low+(high - low) / 2;
    if(arr[mid] == target) return mid;
    elseif(arr[mid] < target) low = mid + 1;
    elsehigh= mid - 1;

}
return low;

} //target is larger than all

public static void main(String[] args) {

    int[] arr={1, 3, 5, 6};
    int target=2;
    System.out.println("Insert/Found at index: " + searchInsert(arr, target));

}
}
```

Time Complexity

Best Case: $O(1)$ (when the target element is found at the middle index in the first comparison)

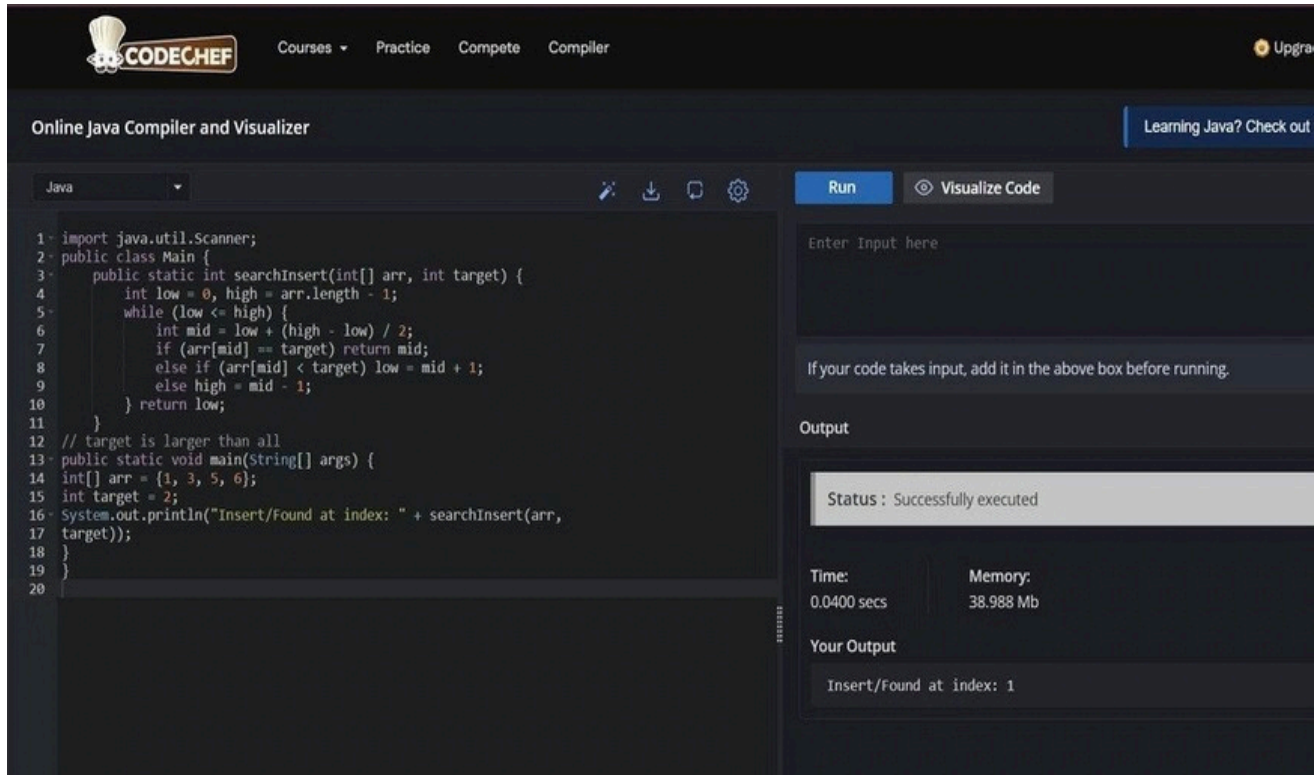
Average Case: $O(\log n)$

Worst Case: $O(\log n)$ (when the search continues until the search space becomes empty)

Space Complexity

$O(1)$ (Constant extra space is used since Binary Search only requires a few variables.)

Input/Output:



Online Java Compiler and Visualizer

Learning Java? Check out

Java

```
1- import java.util.Scanner;
2- public class Main {
3-     public static int searchInsert(int[] arr, int target) {
4-         int low = 0, high = arr.length - 1;
5-         while (low <= high) {
6-             int mid = low + (high - low) / 2;
7-             if (arr[mid] == target) return mid;
8-             else if (arr[mid] < target) low = mid + 1;
9-             else high = mid - 1;
10-        } return low;
11-    }
12-    // target is larger than all
13-    public static void main(String[] args) {
14-        int[] arr = {1, 3, 5, 6};
15-        int target = 2;
16-        System.out.println("Insert/Found at index: " + searchInsert(arr,
17-        target));
18-    }
19- }
20-
```

Run Visualize Code

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0400 secs Memory: 38.988 Mb

Your Output

Insert/Found at index: 1

Practical-3

Aim: To calculate the minimum number of candies required to distribute among N children in a line, ensuring every child gets at least one candy and children with higher ratings receive more candies than their immediate neighbours.

Algorithm:

1. Start the program.
2. Read the number of children *N*.
3. Read the ratings of all children into an array.
4. Create a candy array of size *N* and initialize each element to 1.
5. Traverse the ratings array from **left to right**:
 - o If the current child's rating is greater than the previous child's rating, assign one more candy than the previous child.
6. Traverse the ratings array from **right to left**:
 - o If the current child's rating is greater than the next child's rating, update the candy count to the maximum of its current value and one more than the next child.
7. Calculate the total number of candies by summing all values in the candy array.
8. Display the minimum number of candies required.
9. Stop.

Expected Input/Output (Test Cases)

Test Case	Ratings	Expected	Output
1	[1, 0, 2]	Minimum Candies=	5
2	[1, 2, 2]	Minimum Candies=	4
3	[1, 3, 4, 5, 2]	Minimum Candies=	11

Code Implementation:

```
public class Main {  
    public static int candy(int[] ratings) {  
        int n = ratings.length;  
        int[] candies = new int[n];  
  
        // Give every child 1 candy initially  
        for (int i = 0; i < n; i++) {  
            candies[i] = 1;  
        }  
  
        // Left to Right pass  
        for (int i = 1; i < n; i++) {  
            if (ratings[i] > ratings[i-1]){
```

```
        candies[i] = candies[i - 1] + 1;
    }
}

// Right to Left pass
for (int i = n - 2; i >= 0; i--) {
    if (ratings[i] > ratings[i + 1]) {
        candies[i] = Math.max(candies[i], candies[i + 1] + 1);
    }
}

// Calculate total candies
int total = 0;
for (int candy : candies) {

    total += candy;
}

return total;
}

public static void main(String[] args) {
    int[] ratings = {1, 0, 2};
    System.out.println("Minimum Candies = " + candy(ratings)); // Output: 5
}
}
```

Time Complexity

Best Case: $O(n)$

Average Case: $O(n)$

Worst Case: $O(n)$

Explanation: The algorithm performs two linear traversals of the ratings array and one traversal to calculate the total candies.

Space Complexity

$O(n)$

Explanation: An additional candy array of size N is used to store the number of candies assigned to each child.

Input/Output:

```
1 public class Main {
2
3     public static int candy(int[] ratings) {
4
5         int n = ratings.length;
6
7         int[] candies = new int[n];
8
9         // Give every child 1 candy
10        for (int i = 0; i < n; i++) {
11            candies[i] = 1;
12        }
13
14        // Left to Right
15        for (int i = 1; i < n; i++) {
16
17            if (ratings[i] > ratings[i - 1]) {
18                candies[i] = candies[i - 1] + 1;
19            }
20        }
21
22        // Right to Left
23        for (int i = n - 2; i >= 0; i--) {
24
25            if (ratings[i] > ratings[i + 1]) {
26                candies[i] = Math.max(candies[i], candies[i + 1] + 1);
27            }
28        }
29
30        int total = 0;
31
32        for (int candy : candies) {
33            total += candy;
34        }
35
36        return total;
37    }
38
39    public static void main(String[] args) {
40
41        int[] ratings = {1, 0, 2};
42
43        System.out.println("Minimum Candies = " + candy(ratings));
44    }
45 }
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0400 secs Memory: 38.732 Mb

Your Output

Minimum Candies = 5

Practical-4

Aim: Find the placement of cows that maximizes the minimum distance between any two cows.

Algorithm:

1. Start the program.
2. Read the number of stalls N and the number of cows C .
3. Read the positions of all stalls.
4. Sort the stall positions in ascending order.
5. Initialize:
 - o low = 1 (minimum possible distance)
 - o high = last stall position – first stall position
 - o answer = 0
6. Repeat while low <= high:
 - o Find the middle distance:
 $mid = (low + high) / 2$
 - o Check if all cows can be placed with at least mid distance apart.
 - o If possible:
 - o Store mid as the answer.
 - o Search for a larger distance (low = mid + 1).
 - o Otherwise:
 - o Search for a smaller distance (high = mid - 1).
7. Display the maximum minimum distance (answer).
8. Stop.

Expected Input/Output (Test Cases)

<i>Test Case</i>	<i>Stall Positions</i>	<i>No. of Cows</i>	<i>Expected Output</i>
1	[1, 2, 4, 8, 9] [1,	3	Maximum Minimum Distance = 3
2	2, 4, 8, 9] [1, 3,	2	Maximum Minimum Distance = 8
3	5, 7, 9]	3	Maximum Minimum Distance = 4

Code Implementation:

```
import java.util.Arrays;
public class Main {
    // Greedy helper function to check if cows can be placed with a minimum 'distance'
    static boolean canPlace(int[] stalls, int cows, int distance) {
        int count = 1; // Place the first cow at the first stall
```

```
int lastPlaced = stalls[0];

for (int i = 1; i < stalls.length; i++) {
    if (stalls[i] - lastPlaced >= distance) {
        count++;
        lastPlaced = stalls[i]; // Update the position of the last placed cow
    }
    if (count == cows) {
        return true; // All cows successfully placed
    }
}
return false;
}

public static void main(String[] args) {
    int[] stalls = {1, 2, 4, 8, 9, 12, 16};
    int cows = 3;

    // Step 1: Sort the stall coordinates
    Arrays.sort(stalls);

    // Step 2: Define binary search space boundaries
    int low = 1;
    int high = stalls[stalls.length - 1] - stalls[0];
    int answer = 0;

    // Step 3: Execute Binary Search on Answer
    while (low <= high) {
        int mid = (low + high) / 2;

        if (canPlace(stalls, cows, mid)) {
            answer = mid; // mid is a valid minimum distance, record it
            low = mid + 1; // Try to maximize and look for a larger distance
        } else {
            high = mid - 1; // mid is too large, reduce the distance requirement
        }
    }

    System.out.println("Largest Minimum Distance = " + answer);
}
```

Time Complexity

Sorting the stalls: $O(N \log N)$

Binary Search: $O(\log D)$, where D = maximum stall position – minimum stall position

Feasibility check: $O(N)$ for each binary search iteration.

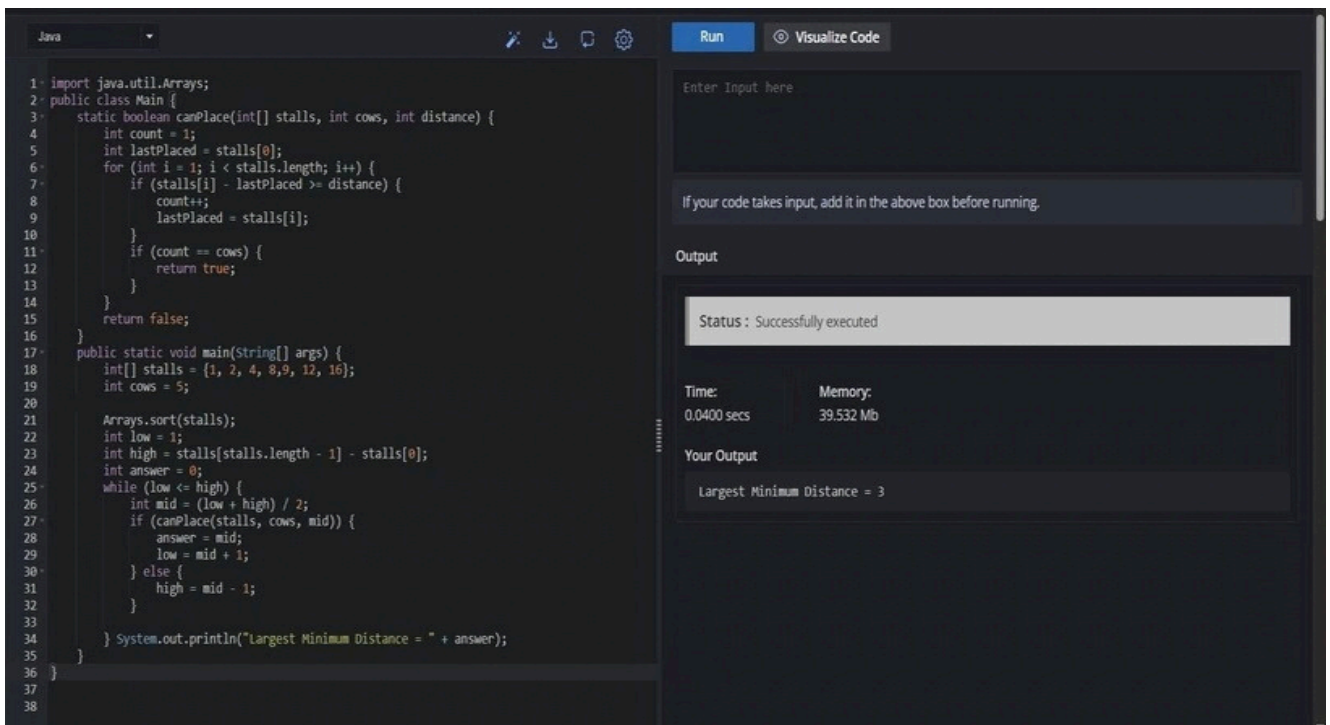
Overall Time Complexity: $O(N \log N + N \log D)$

Space Complexity

$O(1)$ (excluding the space used to store the input array)

Explanation: The algorithm uses only a few extra variables during execution.

Input/Output:



```
1 import java.util.Arrays;
2 public class Main {
3     static boolean canPlace(int[] stalls, int cows, int distance) {
4         int count = 1;
5         int lastPlaced = stalls[0];
6         for (int i = 1; i < stalls.length; i++) {
7             if (stalls[i] - lastPlaced >= distance) {
8                 count++;
9                 lastPlaced = stalls[i];
10            }
11            if (count == cows) {
12                return true;
13            }
14        }
15        return false;
16    }
17    public static void main(String[] args) {
18        int[] stalls = {1, 2, 4, 8, 9, 12, 16};
19        int cows = 5;
20
21        Arrays.sort(stalls);
22        int low = 1;
23        int high = stalls[stalls.length - 1] - stalls[0];
24        int answer = 0;
25        while (low <= high) {
26            int mid = (low + high) / 2;
27            if (canPlace(stalls, cows, mid)) {
28                answer = mid;
29                low = mid + 1;
30            } else {
31                high = mid - 1;
32            }
33        }
34        System.out.println("Largest Minimum Distance = " + answer);
35    }
36 }
37
38
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0400 secs Memory: 39.532 Mb

Your Output

Largest Minimum Distance = 3

Practical-5

Aim: To determine whether a given undirected graph with $|V|$ vertices and $|E|$ edges contains at least one cycle

Algorithm:

1. Start the program.
2. Read the number of vertices V and edges E .
3. Create an adjacency list to represent the graph.
4. Insert all the edges into the adjacency list.
5. Create a **visited[]** array and initialize all vertices as **false**.
6. For each unvisited vertex:
 - o Perform **Depth First Search (DFS)**.
 - o Mark the current vertex as visited.
 - o For every adjacent vertex:
 - If it is not visited, recursively perform DFS.
 - If it is already visited and is **not the parent** of the current vertex, then a **cycle exists**.
7. If a cycle is found, display **"Cycle Detected"**.
8. Otherwise, display **"No Cycle Detected"**.
9. Stop.

Expected Input/Output (Test Cases)

Test Case	Vertices (V)	Edges	Expected Output
1	4	(0-1), (1-2), (2-0), (2-3)	Cycle Detected
2	5	(0-1), (1-2), (2-3), (3-4)	No Cycle Detected
3	3	(0-1), (1-2), (2-0)	Cycle Detected

Code Implementation:

```
import java.util.*;

public class Main {
    static ArrayList<Integer>[] graph;
    static boolean[] visited;

    static boolean dfs(int node, int parent){
        visited[node] = true;
        for (int neighbor : graph[node]){
            if (!visited[neighbor]) {
                if (dfs(neighbor, node))returntrue;
            }
        }
    }
}
```

```
        } else if (neighbor != parent) {
            return true;
        }
    }
    return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int V = sc.nextInt();
    int E = sc.nextInt();

    graph = new ArrayList[V];
    for (int i = 0; i < V; i++) {
        graph[i] = new ArrayList<>();
    }
    for (int i = 0; i < E; i++) {
        int u = sc.nextInt();
        int v = sc.nextInt();
        graph[u].add(v);
        graph[v].add(u);
    }
    visited = new boolean[V];
    boolean cycle = false;

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            // Fixed the syntax error here by changing "-1))" to "-1"
            if (dfs(i, -1)) {
                cycle = true;
                break;
            }
        }
    }

    if (cycle)
        System.out.println("Cycle Detected");
    else
        System.out.println("No Cycle Detected");

    sc.close();
}
```

Time Complexity

Best Case: $O(V + E)$

Average Case: $O(V + E)$

Worst Case: $O(V + E)$

Explanation: Each vertex and each edge is visited at most once during the DFS traversal.

Space Complexity

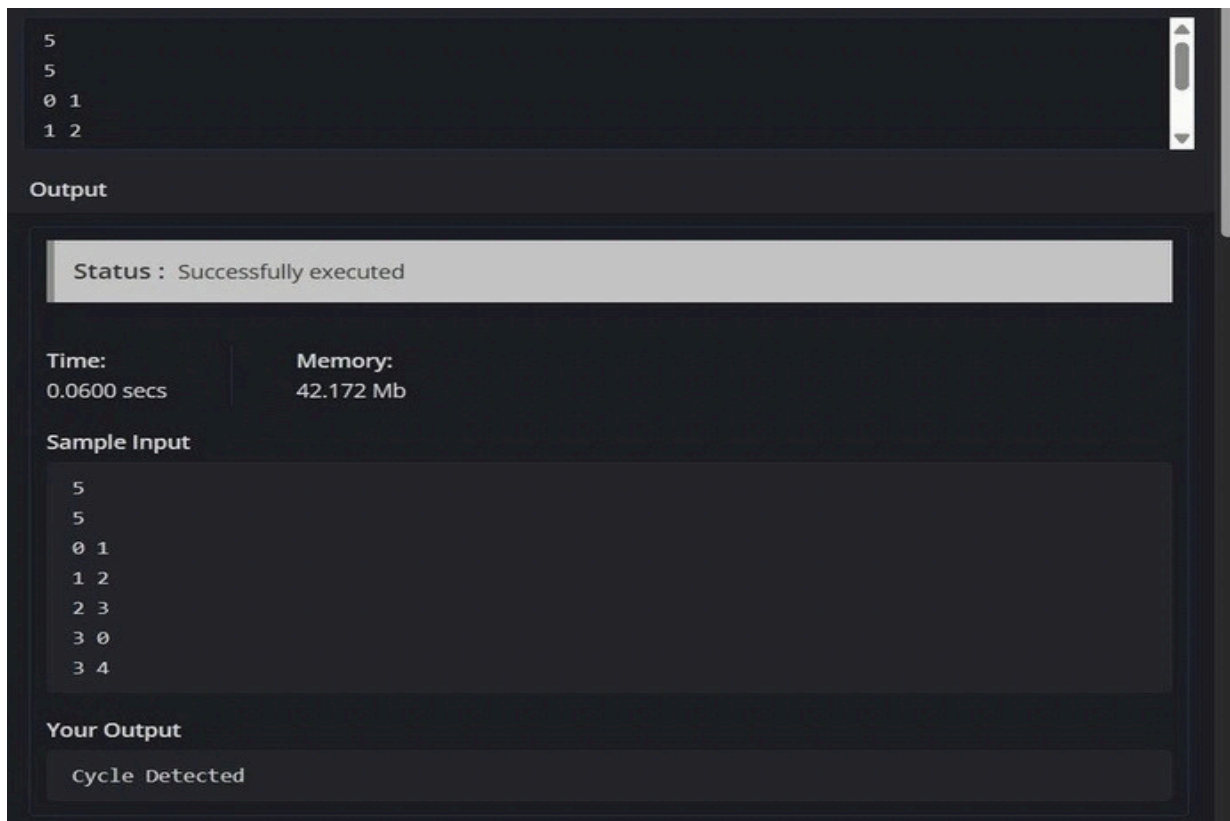
$O(V)$

Explanation: Extra space is required for:

The visited[] array.

The DFS recursion stack (up to V vertices in the worst case).

Input/Output:



```
5
5
0 1
1 2
```

Output

Status : Successfully executed

Time: 0.0600 secs Memory: 42.172 Mb

Sample Input

```
5
5
0 1
1 2
2 3
3 0
3 4
```

Your Output

```
Cycle Detected
```

Practical-6

Aim: To identify and return all critical connections (bridges) in an undirected server network whose removal would disconnect parts of the network

Algorithm: (Using Tarjan's Algorithm)

1. Start the program.
2. Read the number of vertices V and edges E .
3. Create an adjacency list to represent the graph.
4. Initialize the following arrays:
 - o visited[] = false
 - o discovery[] = discovery time of each vertex
 - o low[] = lowest reachable discovery time
 - o parent[] = -1
5. Start DFS from every unvisited vertex.
6. During DFS:
 - o Mark the current vertex as visited.
 - o Assign its discovery and low values.
 - o For every adjacent vertex:
 - If it is not visited:
 - Set its parent.
 - Perform DFS recursively.
 - Update the low value of the current vertex.
 - If low[adjacent] > discovery[current], then the edge (current, adjacent) is a **bridge**.
 - Else if the adjacent vertex is not the parent, update the low value.
7. Store all bridge edges found.
8. Display all critical connections (bridges).
9. Stop.

Expected Input/Output (Test Cases)

<i>Test Case</i>	<i>Vertices (V)</i>	<i>Edges</i>	<i>Expected Output</i>
1	4	(0-1), (1-2), (2-0), (1-3)	Bridges: (1,3)
2	5	(0-1), (1-2), (2-3), (3-4)	Bridges: (0,1), (1,2), (2,3), (3,4)
3	5	(0-1), (1-2), (2-0), (1-3), (3-4)	Bridges: (1,3), (3,4)

Code Implementation:

```
import java.util.*;
public class Main {
    static int V;
    static ArrayList<Integer>[] graph;
```

```
static boolean[] visited;
static int[] disc;
static int[] low;
static int time = 0;
public static void dfs(int node, int parent) {
    visited[node] = true;
    disc[node] = low[node] = time++;
    for (int neighbor : graph[node]) {
        if (neighbor == parent) continue;
        if (!visited[neighbor]) {
            dfs(neighbor, node);
            low[node] = Math.min(low[node], low[neighbor]);
            if (low[neighbor] > disc[node]) {
                System.out.println(node + " - " + neighbor + " is a Bridge");
            }
        } else {
            low[node] = Math.min(low[node], disc[neighbor]);
        }
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    V = sc.nextInt();
    int E = sc.nextInt();
    graph = new ArrayList[V];
    for (int i = 0; i < V; i++) graph[i] = new ArrayList<>();
    for (int i = 0; i < E; i++) {
        int u = sc.nextInt();
        int v = sc.nextInt();
        graph[u].add(v);
        graph[v].add(u);
    }
    visited = new boolean[V];
    disc = new int[V];
    low = new int[V];
    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            dfs(i, -1);
        }
    }
    sc.close();
}
```

Time Complexity***Best Case:*** $O(V + E)$ ***Average Case:*** $O(V + E)$ ***Worst Case:*** $O(V + E)$ ***Explanation:*** Tarjan's Algorithm performs a single DFS traversal, visiting each vertex and edge only once.***Space Complexity*** $O(V + E)$ ***Explanation:*** $O(V)$ for the visited, discovery, low, and parent arrays. $O(E)$ for storing the adjacency list.The recursion stack may use up to $O(V)$ space in the worst case.***Input /Output :***

```
Input
0 1
1 2
2 0
1 3
3 4

Status : Successfully executed

Time: 0.0900 secs    Memory: 42.996 Mb

Sample Input
5 5
0 1
1 2
2 0
1 3
3 4

Your Output
3 - 4 is a Bridge
1 - 3 is a Bridge
```

Practical-7

Aim: To determine the number of islands in a grid of size $N \times M$, where '*I*' represents *Land* and '*0*' represents *water*. An island is formed by connecting adjacent land cells (horizontally or vertically).

Algorithm: (Using Tarjan's Algorithm)

1. Start the program.
2. Read the number of rows N and columns M .
3. Read the grid containing '*I*' (*Land*) and '*0*' (*Water*).
4. Initialize a variable count = 0.
5. Traverse each cell of the grid.
6. If the current cell contains '*I*' and is not visited:
 - o Increment count.
 - o Perform **Depth First Search (DFS)** to visit all connected land cells.
 - o Mark all visited land cells.
7. Continue until all cells are processed.
8. Display the value of count as the number of islands.
9. Stop.

Expected Input/Output (Test Cases)

Test Case	Grid (1 = Land, 0 = Water) {{1,1,0,0}, {1,0,0,1},{0,0,1,1},{0,0,0,0}}	Expected	Output
1	{{1,1,1,1}, {0,0,0,0}}	Number of Islands = 2	
2	{{0,1,0},{1,0,1}} {{1,0,1},{0,0,0}}	Number of Islands = 3	
3	{{1,0,1}}	Number of Islands = 4	

Code Implementation:

```
class NumberOfIslands {
    static void dfs(char[][] grid, int row, int col) {
        if (row < 0 || row >= grid.length || col < 0 || col >= grid[0].length)
            return;

        if (grid[row][col] == '0')
            return;

        grid[row][col] = '0';

        dfs(grid, row - 1, col);
        dfs(grid, row + 1, col);
        dfs(grid, row, col - 1);
        dfs(grid, row, col + 1);
    }
}
```

```
}  
  
static int numIslands(char[][] grid) {  
  
    int count = 0;  
  
    for (int i = 0; i < grid.length; i++) {  
  
        for (int j = 0; j < grid[0].length; j++) {  
  
            if (grid[i][j] == '1') {  
  
                count++;  
  
                dfs(grid, i, j);  
  
            }  
        }  
    }  
  
    return count;  
}  
  
public static void main(String args[]) {  
    char[][] grid = {  
        {'1','1'},  
        {'1','0'}  
    };  
  
    System.out.println("Number of Islands = " + numIslands(grid));  
}  
}
```

Time Complexity***Best Case:*** $O(N \times M)$ ***Average Case:*** $O(N \times M)$ ***Worst Case:*** $O(N \times M)$ ***Explanation:*** Every cell in the grid is visited at most once during the DFS traversal.

Space Complexity

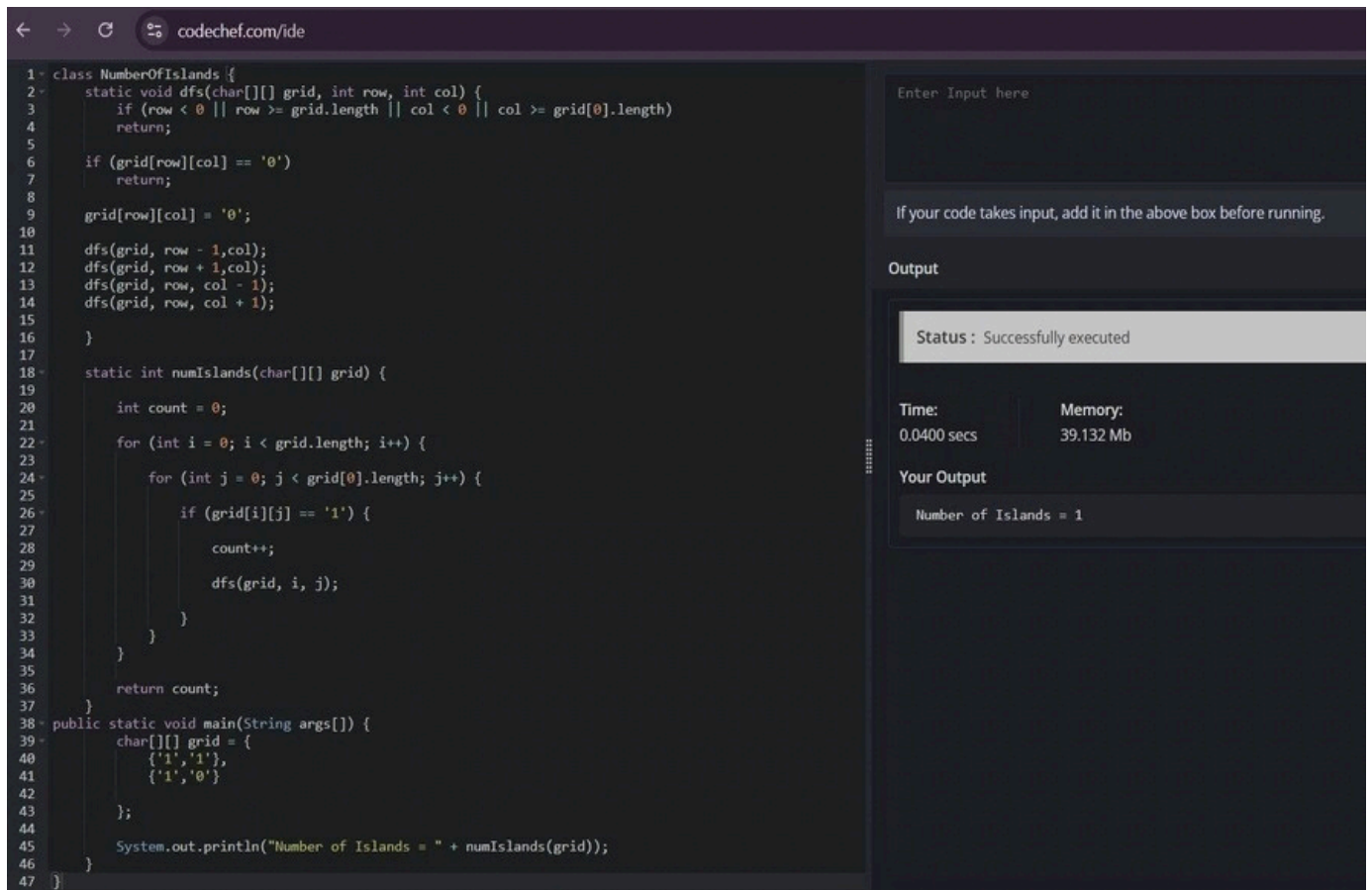
$O(N \times M)$

Explanation:

The *visitedarray* requires $O(N \times M)$ space.

The DFS recursion stack may take up to $O(N \times M)$ space in the worst case when the entire grid is one large island.

Input / Output :



```
1 class NumberOfIslands {
2     static void dfs(char[][] grid, int row, int col) {
3         if (row < 0 || row >= grid.length || col < 0 || col >= grid[0].length)
4             return;
5
6         if (grid[row][col] == '0')
7             return;
8
9         grid[row][col] = '0';
10
11        dfs(grid, row - 1, col);
12        dfs(grid, row + 1, col);
13        dfs(grid, row, col - 1);
14        dfs(grid, row, col + 1);
15    }
16
17    static int numIslands(char[][] grid) {
18        int count = 0;
19
20        for (int i = 0; i < grid.length; i++) {
21            for (int j = 0; j < grid[0].length; j++) {
22                if (grid[i][j] == '1') {
23                    count++;
24                    dfs(grid, i, j);
25                }
26            }
27        }
28
29        return count;
30    }
31
32    public static void main(String args[]) {
33        char[][] grid = {
34            {'1', '1', '1'},
35            {'1', '1', '0'},
36            {'1', '0', '0'}
37        };
38
39        System.out.println("Number of Islands = " + numIslands(grid));
40    }
41 }
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0400 secs Memory: 39.132 Mb

Your Output

Number of Islands = 1